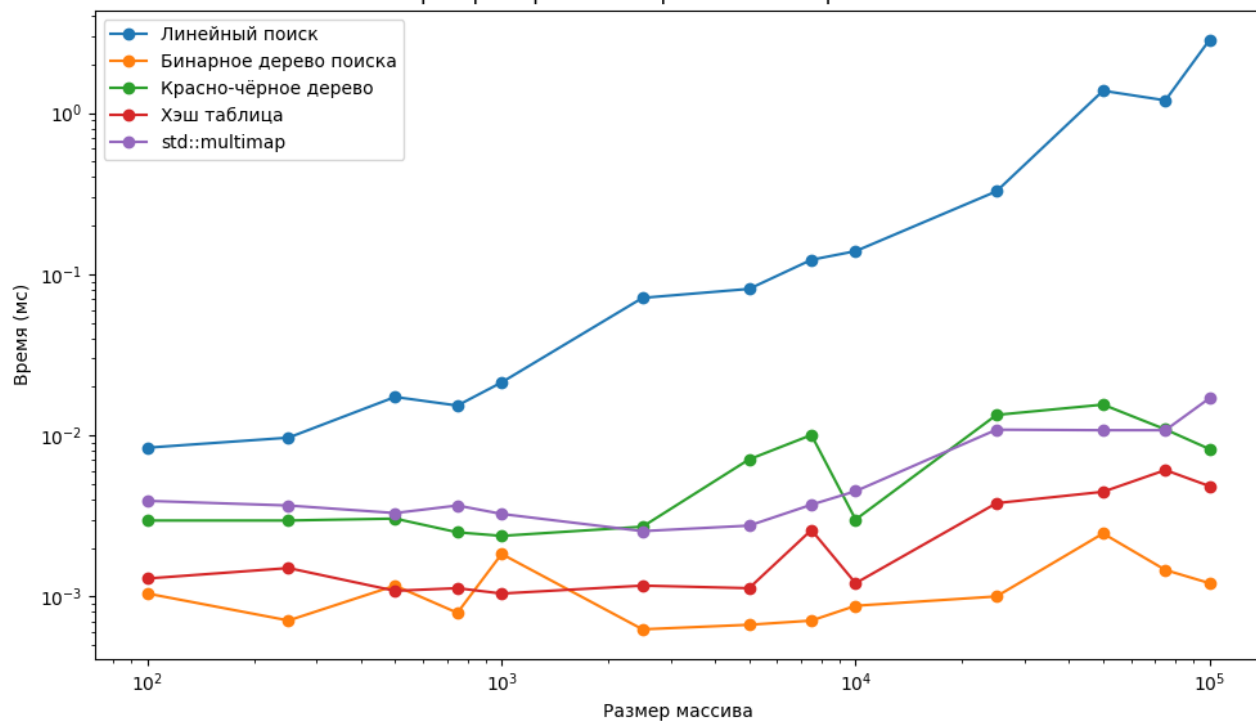
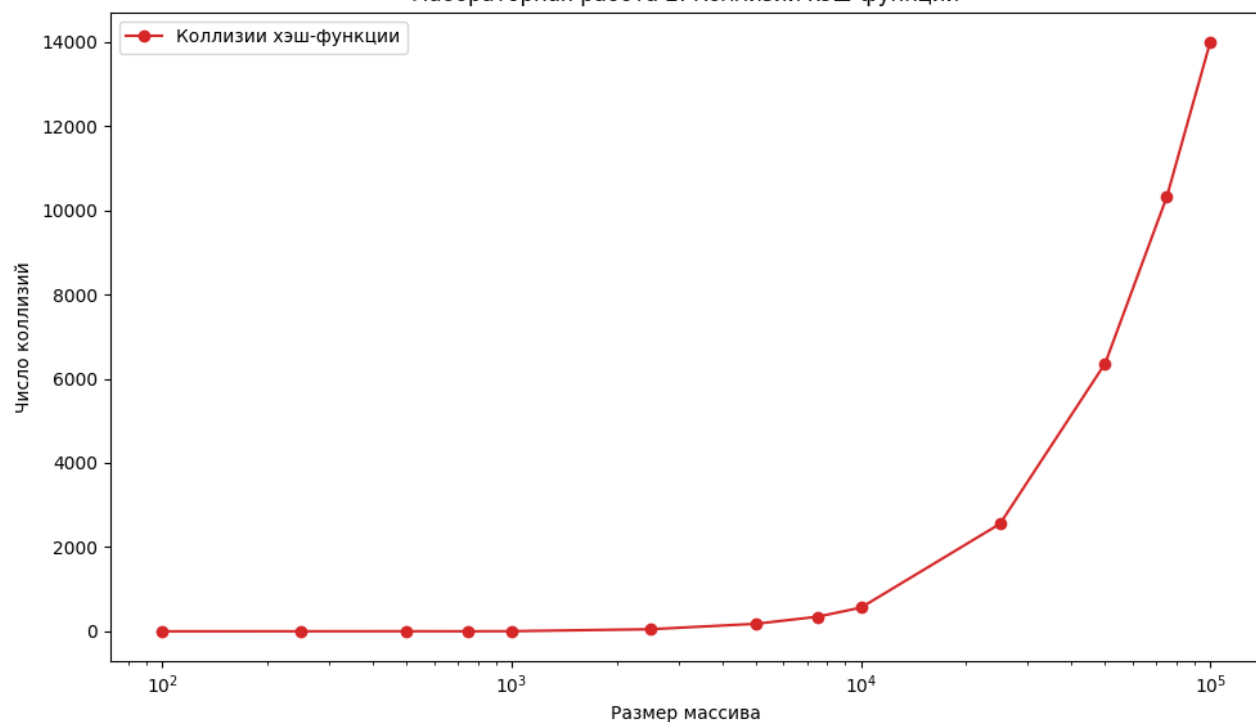


Лабораторная работа 2

Лабораторная работа 2: Сравнение алгоритмов поиска



Лабораторная работа 2: Коллизии хэш-функции



Репозиторий: <https://github.com/diduk001/programming-techniques/tree/main/practice2>

Лабораторная работа 2: Алгоритмы поиска данных

Общее задание

1) Реализовать поиск заданного элемента в массиве (необходимо найти ВСЕ вхождения) объектов по ключу в соответствии с вариантом (структурой данных из ЛР1, ключом является первое НЕ числовое поле объекта) следующими методами:

- линейный поиск
- с помощью бинарного дерева поиска
- с помощью красно-черного дерева
- с помощью хэш таблицы

Все методы необходимо реализовать самостоятельно.

Предполагается, что в исходном массиве данных ключи не уникальны.

2) Для хэш таблицы необходимо реализовать хэш функцию и метод разрешения коллизий. Подсчитать число коллизий хэш функции и построить график зависимости от размерности массива.

3) Выполнить поиск не менее 10 раз на массивах разных размерностей от 100 до 1000000. Засечь (программно) время поиска для всех способов. По полученным точкам построить сравнительные графики зависимости времени поиска от размерности массива.

4) Записать входные данные в ассоциативный массив `multimap` и сравнить время поиска по ключу в нем с временем поиска из п.3. Добавить данные по времени поиска в ассоциативном массиве в общее сравнение с остальными способами и построить график зависимости времени поиска от размерности массива.

5) Требования к отчету аналогичны работе 1.

Вариант 7

Данные: Массив данных о сборной олимпийской команде: ФИО, возраст, рост, вес, вид спорта (сравнение по полям – вид спорта, ФИО, возраст)

Алгоритмы поиска:

- Линейный поиск
- Бинарное дерево поиска
- Красно-чёрное дерево
- Хэш-таблица (разрешение коллизий методом цепочек)
- `std::multimap`

Исходный код

Компилируется с помощью `make build`, скомпилированная программа попадает в `./bin/main`. Сами исходники лежат в `[/src](./src/)`

Документация

Генерируется с помощью `make docs`

- `[Doxyfile](./Doxyfile)`
- `[docs.pdf](./docs.pdf)`

Тестирование

Тестирование происходит на Python в три этапа:

- 1) `make generate` - запускает скрипт `[generate.py](./tests/generate.py)`, который генерирует CSV-файлы разных размеров в `./tests/generated`
- 2) `make test` - запускает скрипт `[test.py](./tests/test.py)`, который запускает `./bin/main` на тестах и записывает результат в `[results/results.csv](./tests/results/results.csv)`
- 3) `make plot` - запускает скрипт `[plot.py](./tests/plot.py)`, который отображает графики на экране и сохраняет их в `[tests/results/](./tests/results/)`

Отчет

Отчет генерируется из ссылки на репозиторий, `[README.md](./README.md)`, графиков и документации скриптом `[report.py](./tests/report.py)` и сохраняется в `[report.pdf](./report.pdf)`

Лабораторная работа №2

Generated by Doxygen 1.17.0

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 BinarySearchTree Struct Reference	5
3.1.1 Detailed Description	5
3.2 BSTNode Struct Reference	6
3.2.1 Detailed Description	6
3.2.2 Member Data Documentation	6
3.2.2.1 left	6
3.2.2.2 originalIndex	6
3.2.2.3 right	6
3.2.2.4 value	7
3.3 FullName Struct Reference	7
3.3.1 Detailed Description	7
3.3.2 Member Function Documentation	8
3.3.2.1 operator"!=()	8
3.3.2.2 operator<()	8
3.3.2.3 operator<=()	8
3.3.2.4 operator==()	9
3.3.2.5 operator>()	9
3.3.2.6 operator>=()	10
3.4 HashTable Struct Reference	10
3.4.1 Detailed Description	11
3.5 RBNode Struct Reference	11
3.5.1 Detailed Description	11
3.5.2 Member Data Documentation	11
3.5.2.1 color	11
3.5.2.2 left	12
3.5.2.3 originalIndex	12
3.5.2.4 parent	12
3.5.2.5 right	12
3.5.2.6 value	12
3.6 RedBlackTree Struct Reference	12
3.6.1 Detailed Description	13
3.7 Sportsman Struct Reference	13
3.7.1 Detailed Description	14
3.7.2 Constructor & Destructor Documentation	14
3.7.2.1 Sportsman()	14

3.7.3 Member Function Documentation	15
3.7.3.1 operator"!=()	15
3.7.3.2 operator<()	15
3.7.3.3 operator<=()	15
3.7.3.4 operator==()	16
3.7.3.5 operator>()	16
3.7.3.6 operator>=()	17
3.7.3.7 to_csv()	17
3.7.4 Member Data Documentation	17
3.7.4.1 age	17
3.7.4.2 full_name	17
3.7.4.3 height_cm	18
3.7.4.4 sport	18
3.7.4.5 weight_kg	18
4 File Documentation	19
4.1 src/search.cpp File Reference	19
4.1.1 Detailed Description	19
4.1.2 Function Documentation	20
4.1.2.1 build_bst()	20
4.1.2.2 build_hash_table()	20
4.1.2.3 build_multimap()	20
4.1.2.4 build_rbt()	21
4.1.2.5 linear_search()	21
4.1.2.6 search_multimap()	22
4.2 src/search.h File Reference	22
4.2.1 Detailed Description	23
4.2.2 Function Documentation	23
4.2.2.1 build_bst()	23
4.2.2.2 build_hash_table()	24
4.2.2.3 build_multimap()	24
4.2.2.4 build_rbt()	24
4.2.2.5 linear_search()	25
4.2.2.6 search_multimap()	25
4.3 search.h	25
4.4 src/sportsman.h File Reference	27
4.4.1 Detailed Description	27
4.5 sportsman.h	27
4.6 src/utlis.h File Reference	29
4.6.1 Detailed Description	30
4.6.2 Function Documentation	30

4.6.2.1 from_csv_file()	30
4.6.2.2 measure_time_ms()	30
4.6.2.3 to_csv_file()	31
4.7 utils.h	31
Index	33

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

BinarySearchTree	Двоичное дерево поиска	5
BSTNode	Узел двоичного дерева поиска	6
FullName	ФИО спортсмена	7
HashTable	Хеш-таблица с разрешением коллизий методом цепочек	10
RBNode	Узел красно-чёрного дерева	11
RedBlackTree	Красно-чёрное дерево	12
Sportsman	Структура данных, описывающая спортсмена	13

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

src/search.cpp	Реализация методов поиска спортсменов в массиве	19
src/search.h	Объявление методов поиска спортсменов в массиве	22
src/sportsman.h	Структура данных, описывающая спортсмена	27
src/utils.h	Вспомогательные функции	29

Chapter 3

Class Documentation

3.1 BinarySearchTree Struct Reference

Двоичное дерево поиска

```
#include <search.h>
```

Public Member Functions

- BinarySearchTree (const [BinarySearchTree](#) &)=delete
- [BinarySearchTree](#) & operator= (const [BinarySearchTree](#) &)=delete
- BinarySearchTree ([BinarySearchTree](#) &&other) noexcept
- [BinarySearchTree](#) & operator= ([BinarySearchTree](#) &&other) noexcept
- void insert (const [Sportsman](#) &value, int index)
- std::vector< int > search (const [Sportsman](#) &target) const

Public Attributes

- [BSTNode](#) * root

3.1.1 Detailed Description

Двоичное дерево поиска

The documentation for this struct was generated from the following files:

- src/[search.h](#)
- src/[search.cpp](#)

3.2 BSTNode Struct Reference

Узел двоичного дерева поиска

```
#include <search.h>
```

Public Member Functions

- BSTNode (const [Sportsman](#) &val, int idx)

Public Attributes

- [Sportsman](#) value
- int [originalIndex](#)
- [BSTNode](#) * [left](#)
- [BSTNode](#) * [right](#)

3.2.1 Detailed Description

Узел двоичного дерева поиска

3.2.2 Member Data Documentation

3.2.2.1 left

```
BSTNode* BSTNode::left
```

Левое поддерево

3.2.2.2 originalIndex

```
int BSTNode::originalIndex
```

Индекс в исходном массиве

3.2.2.3 right

```
BSTNode* BSTNode::right
```

Правое поддерево

3.2.2.4 value

`Sportsman BSTNode::value`

Значение узла

The documentation for this struct was generated from the following files:

- [src/search.h](#)
- [src/search.cpp](#)

3.3 FullName Struct Reference

ФИО спортсмена

```
#include <sportsman.h>
```

Public Member Functions

- `bool operator== (const FullName &other) const`
Проверяет равенство двух ФИО
- `bool operator< (const FullName &other) const`
Сравнивает два ФИО для сортировки (сначала по фамилии, затем по имени, затем по отчеству).
- `bool operator!= (const FullName &other) const`
Проверяет неравенство двух ФИО
- `bool operator> (const FullName &other) const`
Сравнивает два ФИО для сортировки (сначала по фамилии, затем по имени, затем по отчеству).
- `bool operator<= (const FullName &other) const`
Сравнивает два ФИО для сортировки (сначала по фамилии, затем по имени, затем по отчеству).
- `bool operator>= (const FullName &other) const`
Сравнивает два ФИО для сортировки (сначала по фамилии, затем по имени, затем по отчеству).

Public Attributes

- `std::string last_name`
- `std::string first_name`
- `std::string middle_name`

3.3.1 Detailed Description

ФИО спортсмена

3.3.2 Member Function Documentation

3.3.2.1 operator!=(())

```
bool FullName::operator!=(  
    const FullName & other) const [inline]
```

Проверяет неравенство двух ФИО

Parameters

↔	ФИО для сравнения
[other]↔	

Returns

true, если ФИО не совпадают, иначе false

3.3.2.2 operator<()

```
bool FullName::operator<(  
    const FullName & other) const [inline]
```

Сравнивает два ФИО для сортировки (сначала по фамилии, затем по имени, затем по отчеству).

Parameters

↔	ФИО для сравнения
[other]↔	

Returns

true, если текущее ФИО должно идти раньше другого, иначе false

3.3.2.3 operator<=()

```
bool FullName::operator<= (  
    const FullName & other) const [inline]
```

Сравнивает два ФИО для сортировки (сначала по фамилии, затем по имени, затем по отчеству).

Parameters

↔	ФИО для сравнения
[other]↔	

Returns

true, если текущее ФИО должно идти не позже другого, иначе false

3.3.2.4 operator==()

```
bool FullName::operator== (
    const FullName & other) const [inline]
```

Проверяет равенство двух ФИО

Parameters

↔	ФИО для сравнения
[other]↔	

Returns

true, если ФИО совпадают, иначе false

3.3.2.5 operator>()

```
bool FullName::operator> (
    const FullName & other) const [inline]
```

Сравнивает два ФИО для сортировки (сначала по фамилии, затем по имени, затем по отчеству).

Parameters

↔	ФИО для сравнения
[other]↔	

Returns

true, если текущее ФИО должно идти позже другого, иначе false

3.3.2.6 operator>=()

```
bool FullName::operator>= (
    const FullName & other) const [inline]
```

Сравнивает два ФИО для сортировки (сначала по фамилии, затем по имени, затем по отчеству).

Parameters

↔	ФИО для сравнения
[other]↔	

Returns

true, если текущее ФИО должно идти не раньше другого, иначе false

The documentation for this struct was generated from the following file:

- src/[sportsman.h](#)

3.4 HashTable Struct Reference

Хеш-таблица с разрешением коллизий методом цепочек

```
#include <search.h>
```

Public Member Functions

- void insert (const [Sportsman](#) &value, int index)
- std::vector< int > search (const [Sportsman](#) &target) const

Public Attributes

- std::vector< std::vector< std::pair< [Sportsman](#), int > > > buckets
- unsigned int collisions_cnt
- unsigned int total_insertions

Static Public Attributes

- static constexpr int TABLE_SIZE = 65536

3.4.1 Detailed Description

Хеш-таблица с разрешением коллизий методом цепочек

The documentation for this struct was generated from the following files:

- [src/search.h](#)
- [src/search.cpp](#)

3.5 RBNode Struct Reference

Узел красно-чёрного дерева

```
#include <search.h>
```

Public Member Functions

- [RBNode](#) (const [Sportsman](#) &val, int idx)

Public Attributes

- [Sportsman](#) value
- int [originalIndex](#)
- [RBColor](#) color
- [RBNode](#) * left
- [RBNode](#) * right
- [RBNode](#) * parent

3.5.1 Detailed Description

Узел красно-чёрного дерева

3.5.2 Member Data Documentation

3.5.2.1 color

[RBColor](#) [RBNode::color](#)

Цвет узла

3.5.2.2 left

`RBNode* RBNode::left`

Левое поддерево

3.5.2.3 originalIndex

`int RBNode::originalIndex`

Индекс в исходном массиве

3.5.2.4 parent

`RBNode* RBNode::parent`

Родительский узел

3.5.2.5 right

`RBNode* RBNode::right`

Правое поддерево

3.5.2.6 value

`Sportsman RBNode::value`

Значение узла

The documentation for this struct was generated from the following files:

- [src/search.h](#)
- [src/search.cpp](#)

3.6 RedBlackTree Struct Reference

Красно-чёрное дерево

```
#include <search.h>
```

Public Member Functions

- RedBlackTree (const RedBlackTree &)=delete
- RedBlackTree & operator= (const RedBlackTree &)=delete
- RedBlackTree (RedBlackTree &&other) noexcept
- RedBlackTree & operator= (RedBlackTree &&other) noexcept
- void insert (const Sportsman &value, int index)
- std::vector< int > search_all (const Sportsman &target) const

Public Attributes

- RBNode * root

3.6.1 Detailed Description

Красно-чёрное дерево

The documentation for this struct was generated from the following files:

- src/search.h
- src/search.cpp

3.7 Sportsman Struct Reference

Структура данных, описывающая спортсмена

```
#include <sportsman.h>
```

Public Member Functions

- Sportsman (std::string csv_string)
Конструктор, который инициализирует поля структуры на основе строки в формате CSV (вид спорта, фамилия, имя, отчество, возраст, рост, вес).
- std::string to_csv () const
Вывод в строку в формате CSV (вид спорта, фамилия, имя, отчество, возраст, рост, вес).
- bool operator== (const Sportsman &other) const
Проверяет равенство двух спортсменов (сравнение по ФИО).
- bool operator< (const Sportsman &other) const
Сравнивает двух спортсменов для сортировки (сравнение по ФИО).
- bool operator!= (const Sportsman &other) const
Проверяет неравенство двух спортсменов (сравнение по ФИО).
- bool operator> (const Sportsman &other) const
Сравнивает двух спортсменов для сортировки (сравнение по ФИО).
- bool operator<= (const Sportsman &other) const
Сравнивает двух спортсменов для сортировки (сравнение по ФИО).
- bool operator>= (const Sportsman &other) const
Сравнивает двух спортсменов для сортировки (сравнение по ФИО).

Public Attributes

- [FullName](#) `full_name`
- unsigned short `age`
- unsigned short `height_cm`
- unsigned short `weight_kg`
- `std::string` `sport`

3.7.1 Detailed Description

Структура данных, описывающая спортсмена

3.7.2 Constructor & Destructor Documentation

3.7.2.1 Sportsman()

```
Sportsman::Sportsman (  
    std::string csv_string) [inline]
```

Конструктор, который инициализирует поля структуры на основе строки в формате CSV (вид спорта, фамилия, имя, отчество, возраст, рост, вес).

Parameters

<code>[csv_string]</code>	Строка в формате CSV
---------------------------	----------------------

Returns

Инициализированный объект структуры [Sportsman](#)

Exceptions

<code>std::invalid_argument</code> , если	строка не является корректной CSV строкой или содержит некорректные данные
---	--

3.7.3 Member Function Documentation

3.7.3.1 operator!=()

```
bool Sportsman::operator!= (
    const Sportsman & other) const [inline]
```

Проверяет неравенство двух спортсменов (сравнение по ФИО).

Parameters

↔ [other]↔	Спортсмен для сравнения
---------------	-------------------------

Returns

true, если спортсмены не совпадают, иначе false

3.7.3.2 operator<()

```
bool Sportsman::operator< (
    const Sportsman & other) const [inline]
```

Сравнивает двух спортсменов для сортировки (сравнение по ФИО).

Parameters

↔ [other]↔	Спортсмен для сравнения
---------------	-------------------------

Returns

true, если текущий спортсмен должен идти раньше другого, иначе false

3.7.3.3 operator<=()

```
bool Sportsman::operator<= (
    const Sportsman & other) const [inline]
```

Сравнивает двух спортсменов для сортировки (сравнение по ФИО).

Parameters

↔ [other]↔	Спортсмен для сравнения
----------------------------------	-------------------------

Returns

true, если текущий спортсмен должен идти не позже другого, иначе false

3.7.3.4 operator==()

```
bool Sportsman::operator== (
    const Sportsman & other) const [inline]
```

Проверяет равенство двух спортсменов (сравнение по ФИО).

Parameters

↔ [other]↔	Спортсмен для сравнения
----------------------------------	-------------------------

Returns

true, если спортсмены совпадают, иначе false

3.7.3.5 operator>()

```
bool Sportsman::operator> (
    const Sportsman & other) const [inline]
```

Сравнивает двух спортсменов для сортировки (сравнение по ФИО).

Parameters

↔ [other]↔	Спортсмен для сравнения
----------------------------------	-------------------------

Returns

true, если текущий спортсмен должен идти позже другого, иначе false

3.7.3.6 operator>=()

```
bool Sportsman::operator>= (
    const Sportsman & other) const [inline]
```

Сравнивает двух спортсменов для сортировки (сравнение по ФИО).

Parameters

	Спортсмен для сравнения
[other] 	

Returns

true, если текущий спортсмен должен идти не раньше другого, иначе false

3.7.3.7 to_csv()

```
std::string Sportsman::to_csv () const [inline]
```

Вывод в строку в формате CSV (вид спорта, фамилия, имя, отчество, возраст, рост, вес).

Returns

Строка в формате CSV

3.7.4 Member Data Documentation

3.7.4.1 age

```
unsigned short Sportsman::age
```

Возраст спортсмена в годах

3.7.4.2 full_name

```
FullName Sportsman::full_name
```

ФИО спортсмена

3.7.4.3 height_cm

```
unsigned short Sportsman::height_cm
```

Рост спортсмена в сантиметрах

3.7.4.4 sport

```
std::string Sportsman::sport
```

Спорт, которым занимается спортсмен

3.7.4.5 weight_kg

```
unsigned short Sportsman::weight_kg
```

Вес спортсмена в килограммах

The documentation for this struct was generated from the following file:

- [src/sportsman.h](#)

Chapter 4

File Documentation

4.1 src/search.cpp File Reference

Реализация методов поиска спортсменов в массиве

```
#include "search.h"
#include "sportsman.h"
#include <vector>
```

Functions

- `std::vector< int > linear_search` (const `std::vector< Sportsman > &sportsmen`, const `Sportsman &target`)
Линейный поиск всех вхождений `target` в массиве
- `BinarySearchTree build_bst` (const `std::vector< Sportsman > &sportsmen`)
Строит двоичное дерево поиска из массива спортсменов
- `RedBlackTree build_rbt` (const `std::vector< Sportsman > &sportsmen`)
Строит красно-чёрное дерево из массива спортсменов
- `HashTable build_hash_table` (const `std::vector< Sportsman > &sportsmen`, unsigned int &collisions, unsigned int &total_insertions)
Строит хеш-таблицу из массива спортсменов
- `SportsmanMultimap build_multimap` (const `std::vector< Sportsman > &sportsmen`)
Строит ассоциативный массив из вектора спортсменов
- `std::vector< int > search_multimap` (const `SportsmanMultimap &mmap`, const `Sportsman &target`)
Поиск всех вхождений `target` в ассоциативном массиве

4.1.1 Detailed Description

Реализация методов поиска спортсменов в массиве

4.1.2 Function Documentation

4.1.2.1 build_bst()

```
BinarySearchTree build_bst (  
    const std::vector< Sportsman > & sportsmen)
```

Строит двоичное дерево поиска из массива спортсменов

Parameters

sportsmen	Исходный массив
-----------	-----------------

Returns

Построенное дерево

4.1.2.2 build_hash_table()

```
HashTable build_hash_table (  
    const std::vector< Sportsman > & sportsmen,  
    unsigned int & collisions,  
    unsigned int & total_insertions)
```

Строит хеш-таблицу из массива спортсменов

Parameters

sportsmen	Исходный массив
collisions	Число коллизий (выходной параметр)
total_insertions	Общее число вставок (выходной параметр)

Returns

Построенная хеш-таблица

4.1.2.3 build_multimap()

```
SportsmanMultimap build_multimap (  
    const std::vector< Sportsman > & sportsmen)
```

Строит ассоциативный массив из вектора спортсменов

Parameters

sportsmen	Исходный массив
-----------	-----------------

Returns

Построенный ассоциативный массив

4.1.2.4 build_rbt()

```
RedBlackTree build_rbt (  
    const std::vector< Sportsman > & sportsmen)
```

Строит красно-чёрное дерево из массива спортсменов

Parameters

sportsmen	Исходный массив
-----------	-----------------

Returns

Построенное дерево

4.1.2.5 linear_search()

```
std::vector< int > linear_search (  
    const std::vector< Sportsman > & sportsmen,  
    const Sportsman & target)
```

Линейный поиск всех вхождений target в массиве

Parameters

sportsmen	Исходный массив
target	Искомый спортсмен

Returns

Индексы всех найденных вхождений

4.1.2.6 search_multimap()

```
std::vector< int > search_multimap (
    const SportsmanMultimap & mmap,
    const Sportsman & target)
```

Поиск всех вхождений target в ассоциативном массиве

Parameters

mmap	Ассоциативный массив
target	Искомый спортсмен

Returns

Индексы всех найденных вхождений

4.2 src/search.h File Reference

Объявление методов поиска спортсменов в массиве

```
#include "sportsman.h"
#include <map>
#include <vector>
```

Classes

- struct [BSTNode](#)
Узел двоичного дерева поиска
- struct [BinarySearchTree](#)
Двоичное дерево поиска
- struct [RBNode](#)
Узел красно-чёрного дерева
- struct [RedBlackTree](#)
Красно-чёрное дерево
- struct [HashTable](#)
Хеш-таблица с разрешением коллизий методом цепочек

Typedefs

- using SportsmanMultimap = std::multimap<[Sportsman](#), int>

Enumerations

- enum class [RBColor](#) { RED , BLACK }
Цвет узла красно-чёрного дерева

Functions

- `std::vector< int > linear_search` (const `std::vector< Sportsman > &sportsmen`, const `Sportsman &target`)
Линейный поиск всех вхождений `target` в массиве
- `BinarySearchTree build_bst` (const `std::vector< Sportsman > &sportsmen`)
Строит двоичное дерево поиска из массива спортсменов
- `RedBlackTree build_rbt` (const `std::vector< Sportsman > &sportsmen`)
Строит красно-чёрное дерево из массива спортсменов
- `HashTable build_hash_table` (const `std::vector< Sportsman > &sportsmen`, unsigned int `&collisions`, unsigned int `&total_insertions`)
Строит хеш-таблицу из массива спортсменов
- `SportsmanMultimap build_multimap` (const `std::vector< Sportsman > &sportsmen`)
Строит ассоциативный массив из вектора спортсменов
- `std::vector< int > search_multimap` (const `SportsmanMultimap &mmap`, const `Sportsman &target`)
Поиск всех вхождений `target` в ассоциативном массиве

4.2.1 Detailed Description

Объявление методов поиска спортсменов в массиве

4.2.2 Function Documentation

4.2.2.1 build_bst()

```
BinarySearchTree build_bst (
    const std::vector< Sportsman > & sportsmen)
```

Строит двоичное дерево поиска из массива спортсменов

Parameters

sportsmen	Исходный массив
-----------	-----------------

Returns

Построенное дерево

4.2.2.2 build_hash_table()

```
HashTable build_hash_table (  
    const std::vector< Sportsman > & sportsmen,  
    unsigned int & collisions,  
    unsigned int & total_insertions)
```

Строит хеш-таблицу из массива спортсменов

Parameters

sportsmen	Исходный массив
collisions	Число коллизий (выходной параметр)
total_insertions	Общее число вставок (выходной параметр)

Returns

Построенная хеш-таблица

4.2.2.3 build_multimap()

```
SportsmanMultimap build_multimap (  
    const std::vector< Sportsman > & sportsmen)
```

Строит ассоциативный массив из вектора спортсменов

Parameters

sportsmen	Исходный массив
-----------	-----------------

Returns

Построенный ассоциативный массив

4.2.2.4 build_rbt()

```
RedBlackTree build_rbt (  
    const std::vector< Sportsman > & sportsmen)
```

Строит красно-чёрное дерево из массива спортсменов

Parameters

sportsmen	Исходный массив
-----------	-----------------

Returns

Построенное дерево

4.2.2.5 linear_search()

```
std::vector< int > linear_search (
    const std::vector< Sportsman > & sportsmen,
    const Sportsman & target)
```

Линейный поиск всех вхождений target в массиве

Parameters

sportsmen	Исходный массив
target	Искомый спортсмен

Returns

Индексы всех найденных вхождений

4.2.2.6 search_multimap()

```
std::vector< int > search_multimap (
    const SportsmanMultimap & mmap,
    const Sportsman & target)
```

Поиск всех вхождений target в ассоциативном массиве

Parameters

mmap	Ассоциативный массив
target	Искомый спортсмен

Returns

Индексы всех найденных вхождений

4.3 search.h

[Go to the documentation of this file.](#)

```
00001
00005
00006 #pragma once
00007
00008 #include "sportsman.h"
00009 #include <map>
00010 #include <vector>
00011
00012 // — Линейный поиск —
00013
00020 std::vector<int> linear_search(const std::vector<Sportsman> &sportsmen, const Sportsman &target);
00021
00022 // — Двоичное дерево поиска —
00023
```

```

00028 struct BSTNode
00029 {
00030     Sportsman value;
00031     int originalIndex;
00032     BSTNode *left;
00033     BSTNode *right;
00034
00035     BSTNode(const Sportsman &val, int idx);
00036 };
00037
00042 struct BinarySearchTree
00043 {
00044     BSTNode *root;
00045
00046     BinarySearchTree();
00047     BinarySearchTree(const BinarySearchTree &) = delete;
00048     BinarySearchTree &operator=(const BinarySearchTree &) = delete;
00049     BinarySearchTree(BinarySearchTree &&other) noexcept;
00050     BinarySearchTree &operator=(BinarySearchTree &&other) noexcept;
00051     ~BinarySearchTree();
00052
00053     void insert(const Sportsman &value, int index);
00054     std::vector<int> search(const Sportsman &target) const;
00055
00056 private:
00057     BSTNode *insertRec(BSTNode *node, const Sportsman &value, int index);
00058     void searchRec(BSTNode *node, const Sportsman &target, std::vector<int> &indices) const;
00059     static void destroy(BSTNode *node);
00060 };
00061
00067 BinarySearchTree build_bst(const std::vector<Sportsman> &sportsmen);
00068
00069 // — Красно-чёрное дерево —
00070
00075 enum class RBColor { RED, BLACK };
00076
00081 struct RBNode
00082 {
00083     Sportsman value;
00084     int originalIndex;
00085     RBColor color;
00086     RBNode *left;
00087     RBNode *right;
00088     RBNode *parent;
00089
00090     RBNode(const Sportsman &val, int idx);
00091 };
00092
00097 struct RedBlackTree
00098 {
00099     RBNode *root;
00100
00101     RedBlackTree();
00102     RedBlackTree(const RedBlackTree &) = delete;
00103     RedBlackTree &operator=(const RedBlackTree &) = delete;
00104     RedBlackTree(RedBlackTree &&other) noexcept;
00105     RedBlackTree &operator=(RedBlackTree &&other) noexcept;
00106     ~RedBlackTree();
00107
00108     void insert(const Sportsman &value, int index);
00109     std::vector<int> search_all(const Sportsman &target) const;
00110
00111 private:
00112     void rotate_left(RBNode *x);
00113     void rotate_right(RBNode *y);
00114     void fix_insert(RBNode *z);
00115     void search_all_rec(RBNode *node, const Sportsman &target, std::vector<int> &result) const;
00116     static void destroy(RBNode *node);
00117 };
00118
00124 RedBlackTree build_rbt(const std::vector<Sportsman> &sportsmen);
00125
00126 // — Хеш-таблица —
00127
00132 struct HashTable
00133 {
00134     static constexpr int TABLE_SIZE = 65536;
00135     std::vector<std::vector<std::pair<Sportsman, int>>> buckets;
00136     unsigned int collisions_cnt;
00137     unsigned int total_insertions;
00138
00139     HashTable();
00140
00141     void insert(const Sportsman &value, int index);
00142     std::vector<int> search(const Sportsman &target) const;
00143 };
00144

```

```

00152 HashTable build_hash_table(const std::vector<Sportsman> &sportsmen,
00153                             unsigned int &collisions,
00154                             unsigned int &total_insertions);
00155
00156 // — Ассоциативный массив —————
00157
00158 using SportsmanMultimap = std::multimap<Sportsman, int>;
00159
00165 SportsmanMultimap build_multimap(const std::vector<Sportsman> &sportsmen);
00166
00173 std::vector<int> search_multimap(const SportsmanMultimap &mmap, const Sportsman &target);

```

4.4 src/sportsman.h File Reference

Структура данных, описывающая спортсмена

```

#include <stdexcept>
#include <string>

```

Classes

- struct [FullName](#)
ФИО спортсмена
- struct [Sportsman](#)
Структура данных, описывающая спортсмена

4.4.1 Detailed Description

Структура данных, описывающая спортсмена

4.5 sportsman.h

[Go to the documentation of this file.](#)

```

00001
00005
00006 #pragma once
00007
00008 #include <stdexcept>
00009 #include <string>
00010
00015 struct FullName
00016 {
00017     std::string last_name;    /**< Фамилия */
00018     std::string first_name;   /**< Имя */
00019     std::string middle_name;  /**< Отчество */
00020
00026     bool operator==(const FullName &other) const
00027     {
00028         return first_name == other.first_name &&
00029                middle_name == other.middle_name &&
00030                last_name == other.last_name;
00031     }
00032
00038     bool operator<(const FullName &other) const
00039     {
00040         if (last_name != other.last_name)
00041         {

```

```

00042         return last_name < other.last_name;
00043     }
00044     if (first_name != other.first_name)
00045     {
00046         return first_name < other.first_name;
00047     }
00048     return middle_name < other.middle_name;
00049 }
00050
00051 bool operator!=(const FullName &other) const
00052 {
00053     return !(*this == other);
00054 }
00055
00056 bool operator>(const FullName &other) const
00057 {
00058     return other < *this;
00059 }
00060
00061 bool operator<=(const FullName &other) const
00062 {
00063     return !(other < *this);
00064 }
00065
00066 bool operator>=(const FullName &other) const
00067 {
00068     return !(*this < other);
00069 }
00070
00071 };
00072
00073 struct Sportsman
00074 {
00075     FullName full_name;
00076     unsigned short age;
00077     unsigned short height_cm;
00078     unsigned short weight_kg;
00079     std::string sport;
00080
00081     Sportsman(std::string csv_string)
00082     {
00083         int pos = 0;
00084         std::string token;
00085         std::string delimiter = ",";
00086         int field_index = 0;
00087
00088         for (int field_idx = 0; field_idx < 7; field_idx++)
00089         {
00090             pos = csv_string.find(delimiter);
00091             if (pos == std::string::npos)
00092             {
00093                 if (field_idx != 6)
00094                 {
00095                     throw std::invalid_argument("Недостаточно полей в CSV строке");
00096                 }
00097                 token = csv_string;
00098             }
00099             else if (field_idx == 6)
00100             {
00101                 throw std::invalid_argument("Слишком много полей в CSV строке");
00102             }
00103             else
00104             {
00105                 token = csv_string.substr(0, pos);
00106                 csv_string.erase(0, pos + delimiter.length());
00107             }
00108
00109             switch (field_idx)
00110             {
00111             case 0:
00112                 sport = token;
00113                 break;
00114             case 1:
00115                 full_name.last_name = token;
00116                 break;
00117             case 2:
00118                 full_name.first_name = token;
00119                 break;
00120             case 3:
00121                 full_name.middle_name = token;
00122                 break;
00123             case 4:
00124                 age = (unsigned short)std::stoul(token);
00125                 break;
00126             case 5:
00127                 height_cm = (unsigned short)std::stoul(token);
00128                 break;
00129             case 6:

```

```

00159         weight_kg = (unsigned short)std::stoul(token);
00160         break;
00161     }
00162 }
00163 }
00164
00169 std::string to_csv() const
00170 {
00171     return sport + "," +
00172         full_name.last_name + "," +
00173         full_name.first_name + "," +
00174         full_name.middle_name + "," +
00175         std::to_string(age) + "," +
00176         std::to_string(height_cm) + "," +
00177         std::to_string(weight_kg);
00178 }
00179
00185 bool operator==(const Sportsman &other) const
00186 {
00187     return full_name == other.full_name;
00188 }
00189
00195 bool operator<(const Sportsman &other) const
00196 {
00197     return full_name < other.full_name;
00198 }
00199
00205 bool operator!=(const Sportsman &other) const
00206 {
00207     return !(*this == other);
00208 }
00209
00215 bool operator>(const Sportsman &other) const
00216 {
00217     return other < *this;
00218 }
00219
00225 bool operator<=(const Sportsman &other) const
00226 {
00227     return !(other < *this);
00228 }
00229
00235 bool operator>=(const Sportsman &other) const
00236 {
00237     return !(*this < other);
00238 }
00239 };

```

4.6 src/utils.h File Reference

Вспомогательные функции

```

#include "sportsman.h"
#include <chrono>
#include <functional>
#include <iostream>
#include <string>
#include <vector>

```

Functions

- `std::vector< Sportsman > from_csv_file` (const std::string &filename)
Прочитать вектор спортсменов из CSV файла
- `void to_csv_file` (const std::vector< Sportsman > &data, const std::string &filename)
Записать вектор спортсменов в CSV-файл
- `double measure_time_ms` (std::function< std::vector< int >(std::vector< Sportsman > &, const Sportsman &)> find_func, const std::vector< Sportsman > &source, const Sportsman &target)
Измерение времени поиска

4.6.1 Detailed Description

Вспомогательные функции

4.6.2 Function Documentation

4.6.2.1 from_csv_file()

```
std::vector< Sportsman > from_csv_file (
    const std::string & filename)
```

Прочитать вектор спортсменов из CSV файла

Parameters

filename	Путь к входному файлу
----------	-----------------------

Returns

Вектор [Sportsman](#)

4.6.2.2 measure_time_ms()

```
double measure_time_ms (
    std::function< std::vector< int > (std::vector< Sportsman > &, const Sportsman &)>
    find_func,
    const std::vector< Sportsman > & source,
    const Sportsman & target)
```

Измерение времени поиска

Parameters

find_func	Функция поиска (std::vector<int>(std::vector<Sportsman>&))
source	Вектор спортсменов для поиска

Returns

Время поиска (в миллисекундах)

4.6.2.3 to_csv_file()

```
void to_csv_file (
    const std::vector< Sportsman > & data,
    const std::string & filename)
```

Записать вектор спортсменов в CSV-файл

Parameters

data	Вектор <code>Sportsman</code>
filename	Путь к выходному файлу

4.7 utils.h

[Go to the documentation of this file.](#)

```
00001
00005
00006 #pragma once
00007
00008 #include "sportsman.h"
00009 #include <chrono>
00010 #include <functional>
00011 #include <iostream>
00012 #include <string>
00013 #include <vector>
00014
00020 std::vector<Sportsman> from_csv_file(const std::string &filename);
00021
00027 void to_csv_file(const std::vector<Sportsman> &data, const std::string &filename);
00028
00035 double measure_time_ms(std::function<std::vector<int>(std::vector<Sportsman> &, const Sportsman &)>
    find_func, const std::vector<Sportsman> &source, const Sportsman &target);
```


Index

- age
 - Sportsman, [17](#)
- BinarySearchTree, [5](#)
- BSTNode, [6](#)
 - left, [6](#)
 - originalIndex, [6](#)
 - right, [6](#)
 - value, [6](#)
- build_bst
 - search.cpp, [20](#)
 - search.h, [23](#)
- build_hash_table
 - search.cpp, [20](#)
 - search.h, [23](#)
- build_multimap
 - search.cpp, [20](#)
 - search.h, [24](#)
- build_rbt
 - search.cpp, [21](#)
 - search.h, [24](#)
- color
 - RBNode, [11](#)
- from_csv_file
 - utils.h, [30](#)
- full_name
 - Sportsman, [17](#)
- FullName, [7](#)
 - operator!=, [8](#)
 - operator<, [8](#)
 - operator<=, [8](#)
 - operator>, [9](#)
 - operator>=, [9](#)
 - operator==, [9](#)
- HashTable, [10](#)
- height_cm
 - Sportsman, [17](#)
- left
 - BSTNode, [6](#)
 - RBNode, [11](#)
- linear_search
 - search.cpp, [21](#)
 - search.h, [24](#)
- measure_time_ms
 - utils.h, [30](#)
- operator!=
 - FullName, [8](#)
 - Sportsman, [15](#)
- operator<
 - FullName, [8](#)
 - Sportsman, [15](#)
- operator<=
 - FullName, [8](#)
 - Sportsman, [15](#)
- operator>
 - FullName, [9](#)
 - Sportsman, [16](#)
- operator>=
 - FullName, [9](#)
 - Sportsman, [16](#)
- operator==
 - FullName, [9](#)
 - Sportsman, [16](#)
- originalIndex
 - BSTNode, [6](#)
 - RBNode, [12](#)
- parent
 - RBNode, [12](#)
- RBNode, [11](#)
 - color, [11](#)
 - left, [11](#)
 - originalIndex, [12](#)
 - parent, [12](#)
 - right, [12](#)
 - value, [12](#)
- RedBlackTree, [12](#)
- right
 - BSTNode, [6](#)
 - RBNode, [12](#)
- search.cpp
 - build_bst, [20](#)
 - build_hash_table, [20](#)
 - build_multimap, [20](#)
 - build_rbt, [21](#)
 - linear_search, [21](#)
 - search_multimap, [21](#)
- search.h
 - build_bst, [23](#)
 - build_hash_table, [23](#)
 - build_multimap, [24](#)
 - build_rbt, [24](#)
 - linear_search, [24](#)

- search_multimap, [25](#)
- search_multimap
 - search.cpp, [21](#)
 - search.h, [25](#)
- sport
 - Sportsman, [18](#)
- Sportsman, [13](#)
 - age, [17](#)
 - full_name, [17](#)
 - height_cm, [17](#)
 - operator!=, [15](#)
 - operator<, [15](#)
 - operator<=, [15](#)
 - operator>, [16](#)
 - operator>=, [16](#)
 - operator==, [16](#)
 - sport, [18](#)
 - Sportsman, [14](#)
 - to_csv, [17](#)
 - weight_kg, [18](#)
- src/search.cpp, [19](#)
- src/search.h, [22](#), [25](#)
- src/sportsman.h, [27](#)
- src/utils.h, [29](#), [31](#)
- to_csv
 - Sportsman, [17](#)
- to_csv_file
 - utils.h, [30](#)
- utils.h
 - from_csv_file, [30](#)
 - measure_time_ms, [30](#)
 - to_csv_file, [30](#)
- value
 - BSTNode, [6](#)
 - RBNode, [12](#)
- weight_kg
 - Sportsman, [18](#)